1. You can use as much extra memory as you want (but staying within `O(n)` should be your target).
2. The fact that I mentioned linked lists should ring a bell, and make you think about a specific data structure we met in appendix C. Nevertheless, we have seen that it's not enough, if you take it alone.

Both hints above point to the same idea: a single data structure might not be enough to build the most efficient solution to the problem.

On one hand we have data structures that are particularly good for quickly storing and retrieving entries: hash tables are pretty much impossible to beat if that's the game.

On the other hand, hash tables are terrible when it comes to maintain an ordering on things, but we have other structures that handle this very well. Depending on the kind of ordering we would like to keep, we might need trees, or we might be fine with lists. We'll actually see both cases in the rest of the chapter.

In light of this new hint, pause for another minute and try to imagine how to make this work: can you come up with an idea on how to combine a hash table and another data structure in order to optimize all operations on a cache?

7.4.2 Temporal Ordering

Turns out, we have a very simple way to do so. Before delving into this section, you might wat to take a look at appendix C, if you can't remember exactly the running time